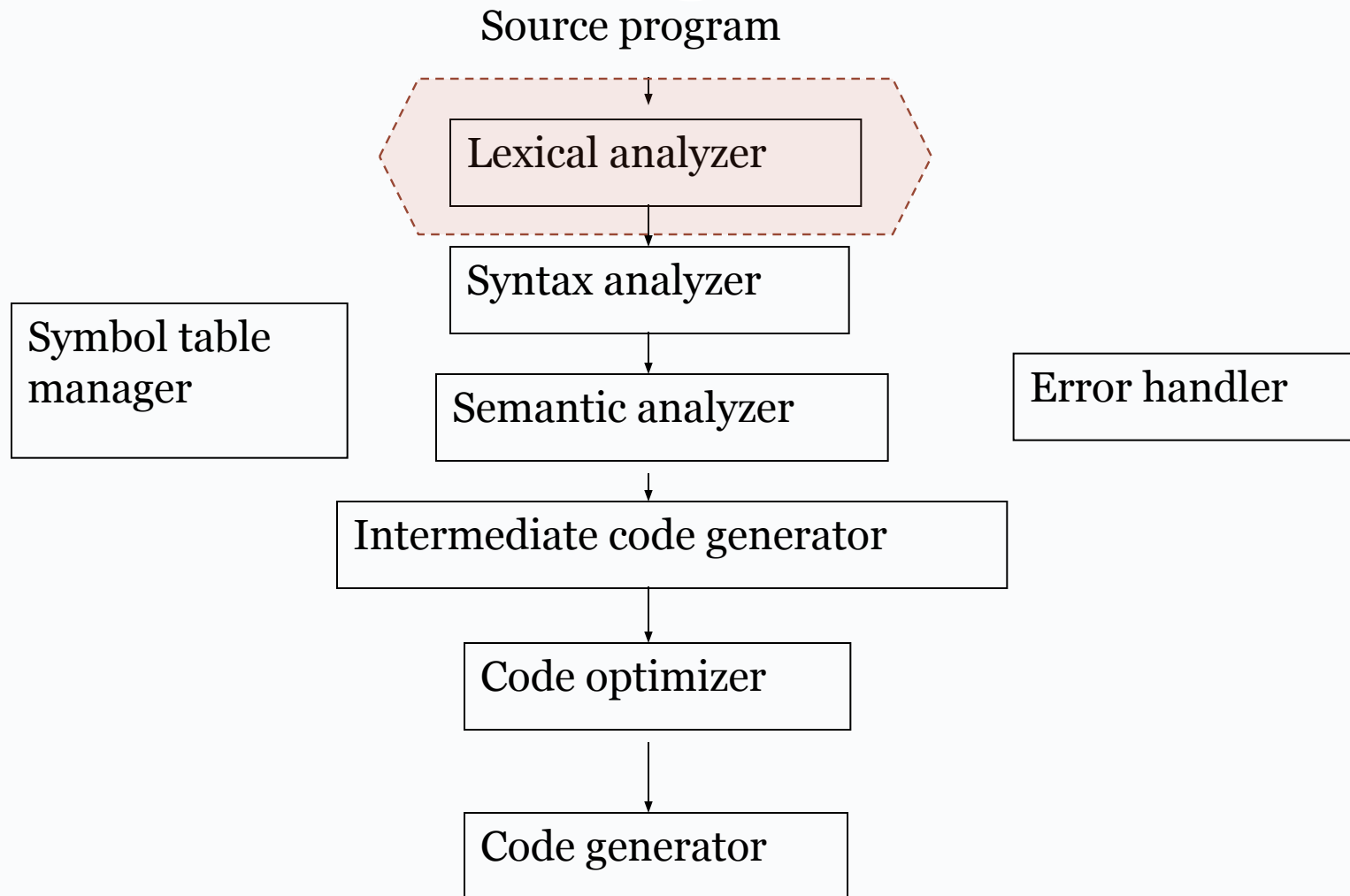


LEXICAL ANALYSIS



Phases of a Compiler

2



The role of the lexical analyzer

3

- First phase of a compiler
- Main task
 - To read the input characters
 - To produce a sequence of **tokens**
 - used by the parser for syntax analysis
 - An assistant of parser

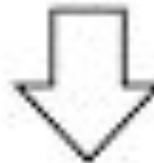
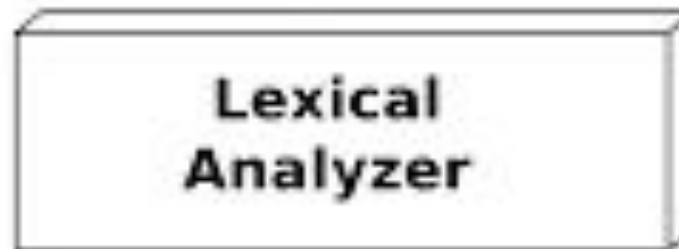
The role of the lexical analyzer

4

i f { x > 3 . 1



Character Stream



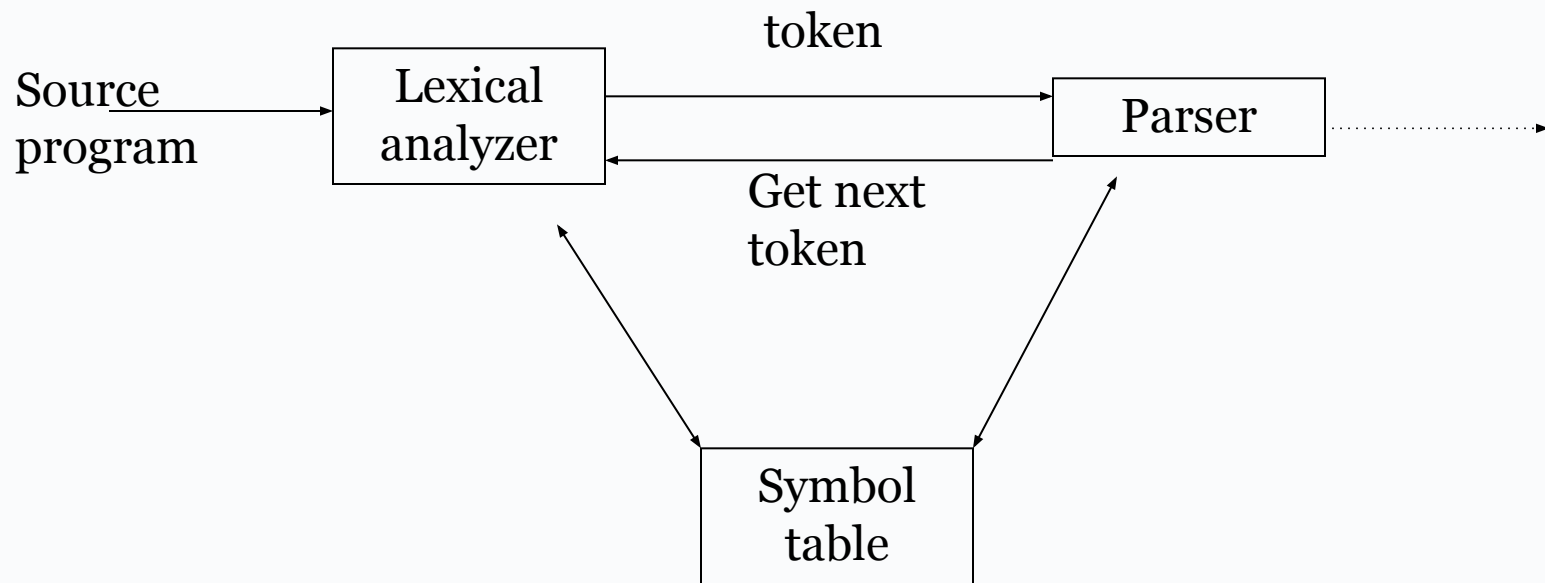
Token Stream

KEYWORD	BRACKET	IDENTIFIER	OPERATOR	NUMBER
"if"	"{"	"x"	">"	"3.1"

The role of the lexical analyzer

5

Interaction of lexical analyzer with parser



The role of the lexical analyzer

6

Processes of lexical analyzers: a cascade of two parts

- Scanning
 - Pre-processing
 - Strip out comments and white space
 - Macro functions (replace/take action)
 - Correlating error messages from compiler with source program
 - A line number can be associated with an error message
- Lexical analysis

The role of the lexical analyzer

7

Terms of the lexical analyzer

○ Token

- Types of words in source program
 - Keywords, operators,
 - identifiers, constants,
 - literal strings,
 - punctuation symbols (such as commas, semicolons)

○ Lexeme

- Actual words in source program matched by pattern
 - valid token

○ Pattern

- A rule describing the set of lexemes that can represent a particular token in source program
- Relation $\{<.<=,>,>=,==,<>\}$

The role of the lexical analyzer

8

Example

Token	Informal description	Sample lexemes
if	Characters i, f	if
else	Characters e, l, s, e	else
relation	< or > or <= or >= or == or !=	<=, !=
id	Letter followed by letter and digits	pi, score, D2
number	Any numeric constant	3.14159, 0, 6.02e23
literal	Anything but “ sorrounded by “	“core dumped”

The role of the lexical analyzer

9

Attributes for Tokens

- When more than one pattern matches a lexeme, the lexical analyzer must provide additional information about the lexeme
- The token influences parsing decision and attribute influences translation of tokens
- Token has single attribute namely pointer to the symbol table entry where information about the token is kept.

The role of the lexical analyzer

10

E.g $E = M * C ** 2$ (written in FORTRAN)

<**id**, pointer to symbol-table entry for E>

<**assign_op**,>

<**id**, pointer to symbol-table entry for M>

<**multi_op**,>

<**id**, pointer to symbol-table entry for C>

<**exp_op**,>

<**num**, integer value 2>

Specification of Tokens

11

- **Regular expressions** are an important notation for specifying patterns.
- Each **pattern matches** a set of strings, so regular expressions will serve as names for sets of strings.

Regular Definition of Tokens

Defined in regular expression

e.g. Id $\rightarrow$ **letter(letter|digit)**

letter $\rightarrow$ A|B|...|Z|a|b|...|z

digit $\rightarrow$ 0|1|2|...|9

Specification of Tokens

12

● Some definitions

- *alphabet* : a finite set of symbols. E.g. {a, b, c}
- A *string* over an alphabet is a finite sequence of symbols drawn from that alphabet (sometimes a string is also called a sentence or a word).
- A *language* is a set of strings over an alphabet.
- Operation on languages (a set):
 - union of L and M, $L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
 - concatenation of L and M
$$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$$
 - Kleene closure of L, $L^* = \bigcup_{i=0}^{\infty} L^i$
 - Positive closure of L, $L^+ = \bigcup_{i=1}^{\infty} L^i$

Specification of Tokens

13

Notational Short-hands

a) One or more instances

$(r)^+ \text{ digit}^+$

b) Zero or one instance

$r?$ is a shorthand for $r|\epsilon$ $(E(+|-)?\text{digits})?$

c) Character classes

$[a-z]$ denotes $a|b|c|\dots|z$

$[A-Za-z] [A-Za-z0-9]$

SPECIAL CHARACTERS

Metacharacter	Description
*	Zero or more
+	One or more
?	Zero or one; Not greedy
^	Beginning of string
\$	End of string
.	Any
[]	Group
{ }	Marks the start and end of a quantifier expression
()	Capture
	or
\	Escape

METACHARACTERS

Metacharacter	Description
\b	Word boundary
\B	Non word boundary
\d	Decimal digit Equivalent to [0-9]
\D	Non decimal digit Equivalent to [^0-9]
\w	Word Equivalent to [a-zA-Z_0-9]
\W	Non word Equivalent to [^a-zA-Z_0-9]
[xyz]	Group
[^xyz]	Negated group
[a-z]	Group

METACHARACTERS

Metacharacter	Description
[^a-z]	Negated group
{n}	Exactly N
{n,}	At least N
{n,m}	From N to M
(pattern)	Capture
(?<name>pattern)	Named capture
(?:pattern)	Non capture
(?=pattern)	Positive lookahead
(?!pattern)	Negative lookahead
(?<=pattern)	Positive lookbehind
(?<!=pattern)	Negative lookbehind
\cx	CTRL char
\xn	ASCII value

NONPRINTING CHARACTERS

Metacharacter	Description
\f	Form feed Equivalent to \x0c\cL
\n	New line Equivalent to \x0a\cJ
\r	Carriage return Equivalent to \x0d\cM
\s	Whitespace Equivalent to [\f\n\r\t\v]
\S	Non whitespace Equivalent to [^\f\n\r\t\v]
\t	Tab Equivalent to \x09\cI
\v	Vertical tab Equivalent to \x0b\cK

ORDER OF PRECEDENCE

Operator or operators	Description
\	Escape
(), (?:), (?:=), []	Parentheses and brackets
*, +, ?, {n}, {n,}, {n,m}	Quantifiers
^, \$, \anymetacharacter	Anchors and sequences
	Alternation

Expressions – Word and Ranges

- ABC
 - *Word equals to ABC*
- [a-z]
 - *Matching lowercase alphabets eg. a, b, c, d, ..., x, y, z*
- [A-Z]
 - *Matching uppercase alphabets eg. A, B, C, D, ..., X, Y, Z*
- [0-9]
 - *Matching digits eg. 0, 1, 2, ..., 8, 9*

Expressions – Words with size

- $[a-z]^+$
 - Any word containing all alphabets excluding null
 - eg. aaaa, abc, owais, house ...
- $[A-Z]^*$
 - Any word containing all alphabets including null
- $[A-Za-z]^*$
 - Any word containing upper and lower case alphabets
 - Eg. Owais, House, house...
- $[A-Za-z0-9]\{5\}$
 - Word containing any alphabet and number with word of size 6
 - Eg. abcde, Owais, abc12, 6011...
- $[A-Za-z\d]\{3, 8\}$
 - Word of size ranging from 3 to 8

Expression – String matching

- (admin|manager)
 - *String equal to admin or manager*
- (mon|tues|wednes|thurs|fri|satur|sun)day
 - *Matching week days*
- ^(math|calculus)\$
 - *Starting and ending or exactly math or calculus*
- ^(math|calculus)
 - *Starting with word math or calculus*
 - *Eg math is a subject.*

Username RegEx

- Size ranging from 3 to 12
 - $[a-z0-9]\{3, 12\}$
- Can contain small alphabets and digits
- Expression
 - $[a-z][a-z0-9]\{2, 11\}$

Password RegEx

- Size greater than 8
- Contain alphabet and digits
- Expression
 - `[a-zA-Z0-9]{8,}`
- Can contain special character
 - `[a-zA-Z0-9@#^%]{8,}`

Email Address RegEx

- Contains @ and .
- Contains host eg gmail.com, pia.aero, github.io
- Contains username eg. P146011
 - *Range 4 to 24*
- Expression
 - `[a-zA-Z0-9]{4,24}@[a-z0-9\.-]\.[a-z]{2,4}`
 - *Work for most email.*
- Dot mean "Any thing" in regex
 - *.a mean ending with a of size 2 eg, aa, ab, %a, ga...*
 - *A.*B mean starting with A and ending with B*

Validate Date

- 31-11-1999
 - Expression: $[0-9]\{1,2\}-[0-9][1,2]-[0-9]\{4\}$
 - Validates: 1-1-2000, 07-10-2016 ...
 - Problem...
 - $0[0-9]1[12]$
 - for year
 - $0[1-9][12][0-9]3[01]$
 - for month
 - $(19|20)[0-9]\{2\}$ from year ranging
 - 1900-2099

Where is it used?

- Strong password validation
- Login via email or phone in Facebook
- Google Search Operators
 - *define: abracadabra*
 - *#soachishti -> Find hashtags*
 - *Made by * -> Unknown or wildcard terms.*
- Spam/Junk filter in email
 - *You won million dollars...*
- Data scraping
 - *Extracting name and email from websites*
- Text Processing
 - *Remove duplicate sentences*
 - *Remove slang*

Recognition of Tokens

27

Grammar:

$stmt \rightarrow |if\ expr\ then\ stmt$
 $|if\ expr\ then\ stmt\ else\ stmt$
 $| \in$
 $expr \rightarrow term\ relop\ term\ | \ term$
 $term \rightarrow id\ | \ num$

Assume Following Tokens:

if, then, else, relop, id, num

Regular definitions

if $\leftarrow$ if

then $\leftarrow$ then

else $\leftarrow$ else

relop $\leftarrow$ < | <= | > | >= | = | <>

id $\leftarrow$ letter (letter | digit)*

num $\leftarrow$ digit⁺ (. digit⁺) ? (E(+ | -) ? digit⁺) ?

Recognition of Tokens

28

Task of recognition of token in a lexical analyzer

- Isolate the lexeme for the next token
- Produce as output a pair consisting of the appropriate token and attribute-value, such as <id, pointer to table entry>

Recognition of Tokens

29

Task of recognition of token in a lexical analyzer

Regular expression	Token	Attribute-value
if	if	-
id	id	Pointer to table entry
<	relop	LT

Recognition of Tokens

30

Regular Expression	Token	Attribute-Value
WS	-	-
if	if	-
then	then	-
else	else	-
id	id	pointer to table entry
num	num	pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
< >	relop	NE
>	relop	GT
>=	relop	GE

Example: Lexeme(1/2)

31

- Divide the following C++ program into appropriate lexemes.

```
float limitedSquare(x)  
{ float x  
  return (x <= -10.0 || x >= 10.0) ? 100 : x*x;  
}
```

Answer

```
<float> <id, limitedSquaare> <(> <id, x> <)> <{> <float> <id, x> <return> <(>  
<id, x> <op, "<="> <num, -10.0> <op, "||"> <id, x> <op, ">="> <num, 10.0> <)>  
<op, "?"> <num, 100> <op, ":"> <id, x> <op, "*"> <id, x> <}>
```

Example: Lexeme(2/2)

32

Tagged languages like HTML or XML are different from conventional programming languages in that the tags are either very numerous (as in HTML) or a user-definable set (as in XML). Further, tags can often have parameters. Suggest and divide the following HTML document into appropriate lexemes.

*Here is a photo of my house; <p>
see More Picture if you liked that one.</p>*

Answer

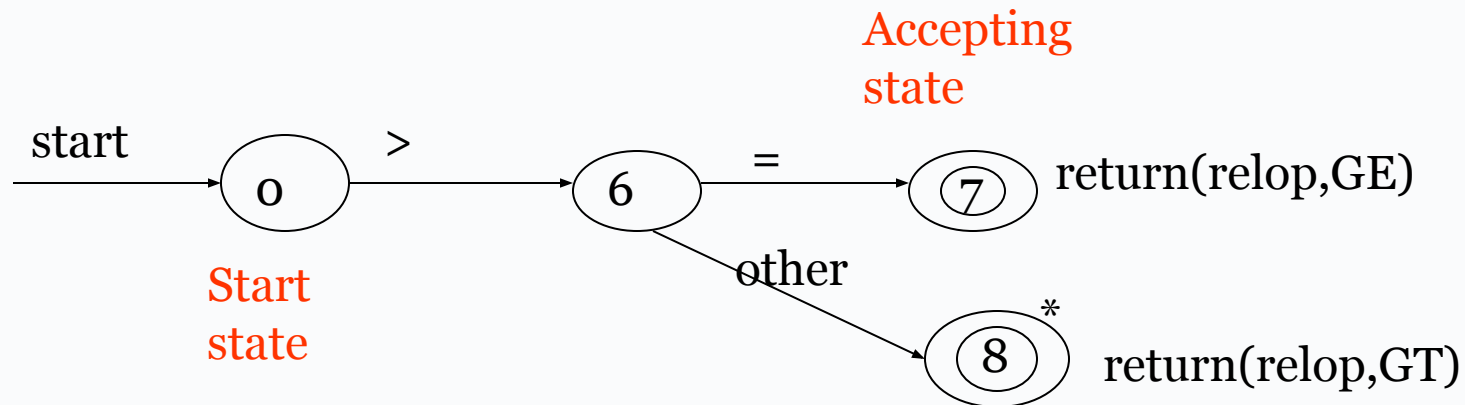
<text, "Here is a photo of"> <nodestart, b> <text, "my house"> <nodeend, b>
<nodestart, p> <selfendnode, img> <selfendnode, br> <text, "see"> <nodestart, a>
<text, "More Picture"> <nodeend, a> <text, "if you liked that one."> <nodeend, p>

Recognition of Tokens

33

Methods to recognition of token

- Use Transition Diagram

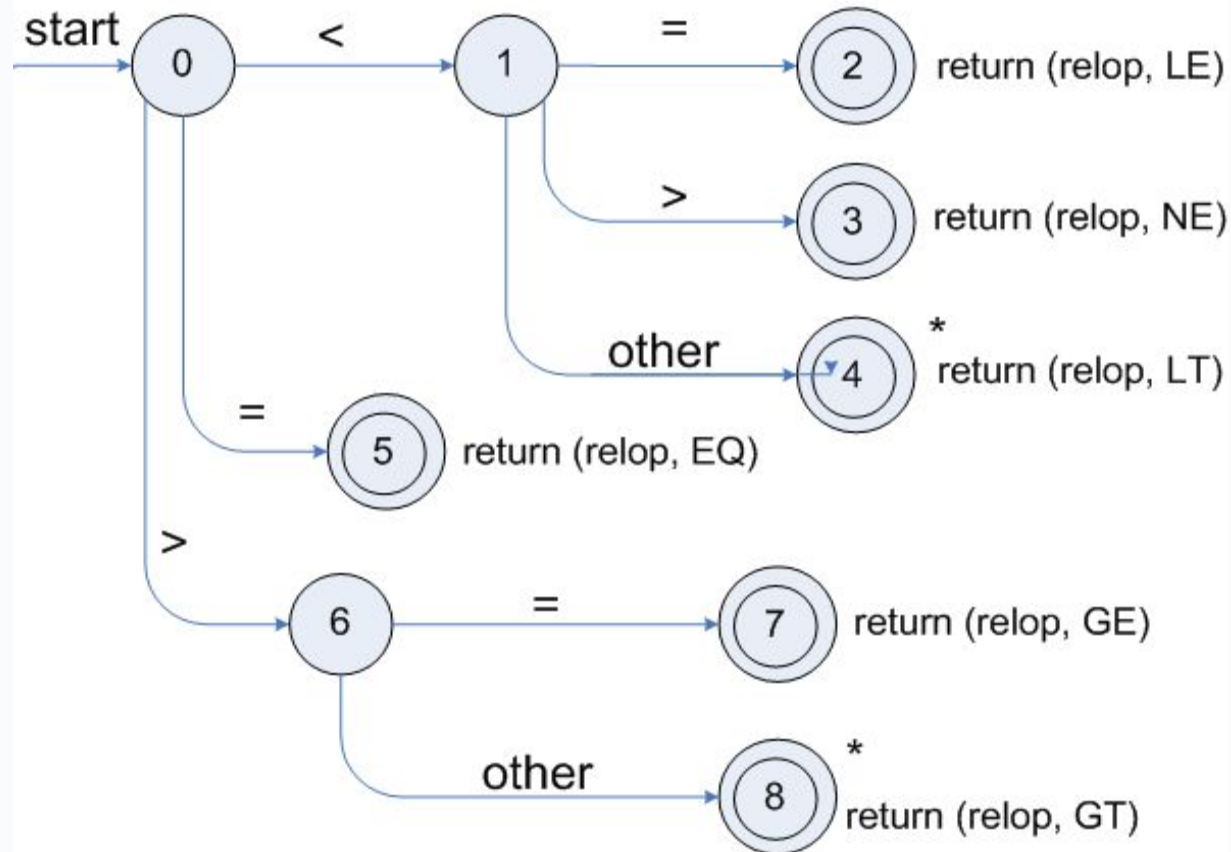


Transition diagram for `>=`

Recognition of Tokens

34

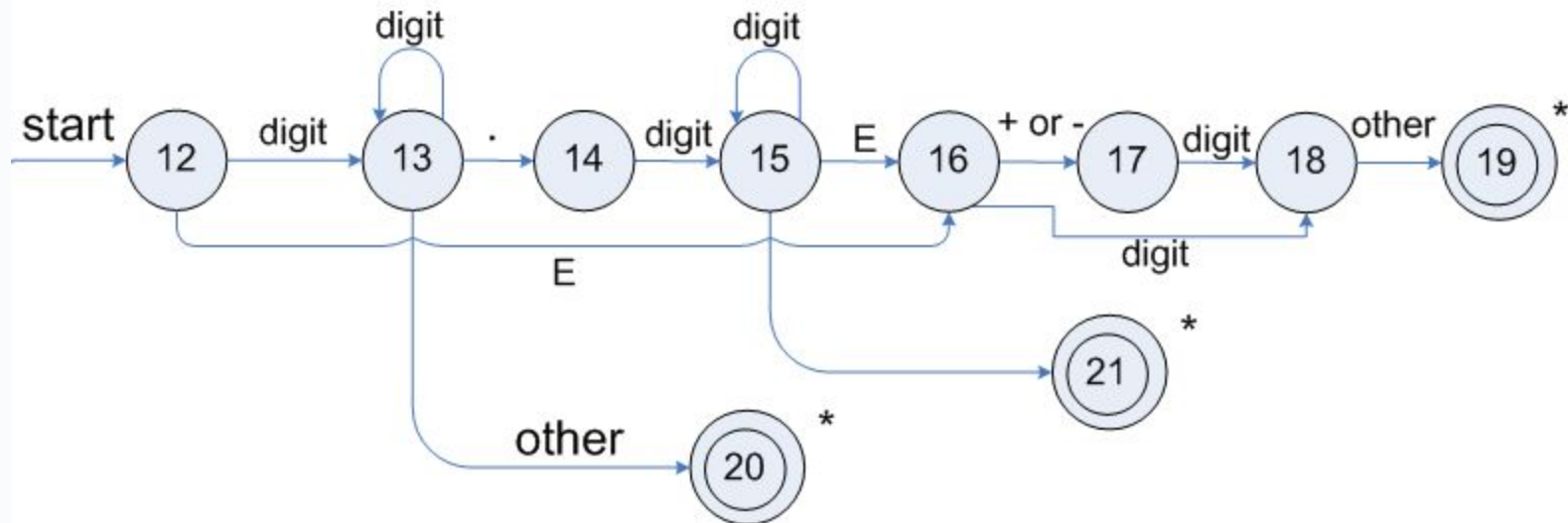
Transition diagram for relop



Recognition of Tokens

35

- Transition diagram for unsigned numbers



FLEX

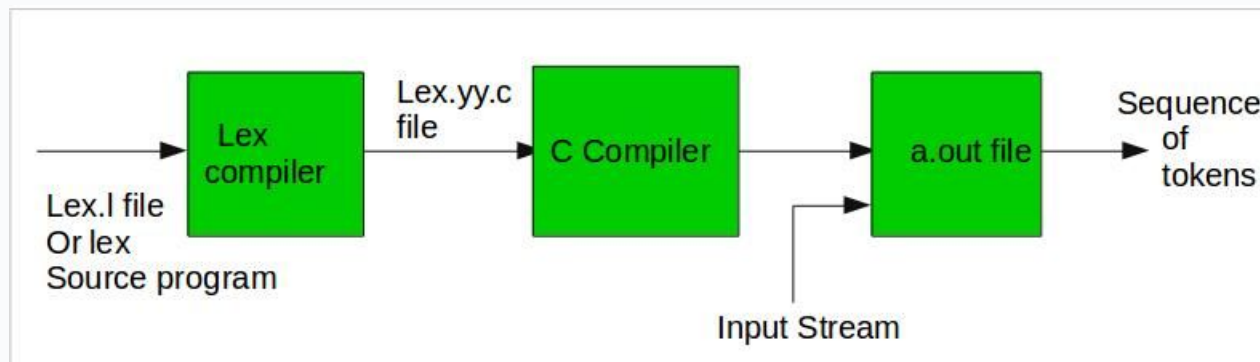
36

- **Lexical Analysis:** Dividing the input into meaningful units. For a C program the units are *variables, constants, keywords, operators, punctuation* etc. These units also called as tokens.
Flex: A fast Lexical Analyzer Generator
- **Parsing:** Involves finding the relationship between input tokens. For a C program, one needs to identify valid expressions, statements, blocks, procedures etc. (Use **Bison**)

FLEX

37

FLEX (fast lexical analyzer generator) is a tool/computer program for generating lexical analyzers (scanners or lexers)



Flex

38

- Flex is a tool for generating scanners.
- Flex source is a table of regular expressions and corresponding program fragments.
- Generates `lex.yy.c` which defines a routine `yylex()`

Flex



.l file $\Rightarrow$ lex $\Rightarrow$ lex.yy.c

lex.yy.c and other files $\Rightarrow$ gcc $\Rightarrow$ lexical analyzer

input stream $\Rightarrow$ lexical analyzer $\Rightarrow$ actions taken
when rules applied

Thank You